

A Framework for Database Search with AI Models in Mass Spectrometry-Based Proteomics

Konstantinos Kalogeropoulos,* Jeroen Van Goey, Timothy P. Jenkins, and Kevin Michael Eloff

Cite This: *J. Proteome Res.* 2026, 25, 2234–2242

Read Online

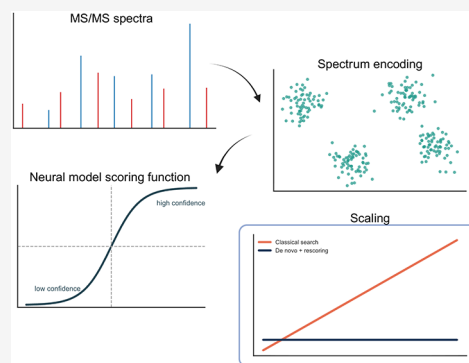
ACCESS |

Metrics & More

Article Recommendations

ABSTRACT: Database search remains the primary strategy for peptide detection in mass spectrometry-based proteomics, but growing data sets and increasingly expansive peptide search spaces now challenge its computational limits. At the same time, machine learning has transformed multiple aspects of spectrum identification and is increasingly applied directly to peptide-spectrum matching. Neural network models have been proposed as core engines for database search, yet the computational complexities of such approaches have not been systematically defined or compared. Here, we present a range of emerging approaches for database search and a theoretical framework for runtime and scaling in spectrum identification, contrasting classical search strategies with emerging neural network-based methods. We analyze asymptotic complexity in the number of spectra and peptide candidates and estimate practical wall time and memory requirements under realistic hardware assumptions. Our framework highlights trade-offs and provides a guide for selecting and developing scalable peptide search strategies in the era of large models and proteomics data sets. We therefore consider whether learned scoring models may progressively replace or augment classical similarity functions at the peptide–spectrum scoring level.

KEYWORDS: proteomics, mass spectrometry, database search, *de novo* peptide sequencing, peptide spectrum match, reference database, machine learning, AI models, hybrid proteomics searches



INTRODUCTION

Peptide detection via reference database lookup is a cornerstone of bottom-up, mass spectrometry-based proteomics.^{1,2} Traditionally, this task has been performed by search engines that compare experimental tandem mass spectra to theoretical fragments of peptides derived from *in silico* digestion of a protein database. Candidate peptides are selected based on precursor mass, scored against observed fragment ions, and subsequently refined through machine learning-based rescoring strategies.^{3–5} This workflow has been extensively optimized and remains highly effective, but the rapid growth of data volumes, the expansion of search spaces, and the emergence of new application areas now challenge its computational limits.⁶ Modern data sets routinely comprise millions of spectra, while peptide databases can reach hundreds of millions of entries when multiple organisms, proteogenomic variants, or variable modifications are included.^{6–9}

Machine learning has already transformed several aspects of spectrum identification.¹⁰ Deep learning models are widely used in peptide spectrum prediction to generate synthetic libraries, and in direct scoring of candidate matches as implemented in popular tools for peptide retention and fragment ion prediction,^{11–14} as well as peptide-spectrum match (PSM) assignment and rescoring.^{15–17} These advances

motivate the question of whether large neural architectures could be used as the core computational engine for database search itself. Early proposals along these lines have shown promise,^{17–20} yet there is no unified framework for understanding how such approaches scale computationally. Although their per-comparison cost is orders of magnitude higher than classical scoring, under certain conditions they may remain tractable or even advantageous. Such approaches can potentially enable improvements in accuracy, calibration, or efficiency as database sizes grow.^{21,22}

In this work, we present a comparative framework for evaluating both current and emerging strategies for spectrum identification. Building on advances in joint spectrum–peptide embeddings,^{20,23,24} encoder–decoder models for *de novo* sequencing,^{25–27} and inspired by recent proposals for database search with deep learning models,^{18,28} we formalize how such methods can be implemented and compare their scaling

Received: November 21, 2025**Revised:** February 27, 2026**Accepted:** March 25, 2026**Published:** March 30, 2026

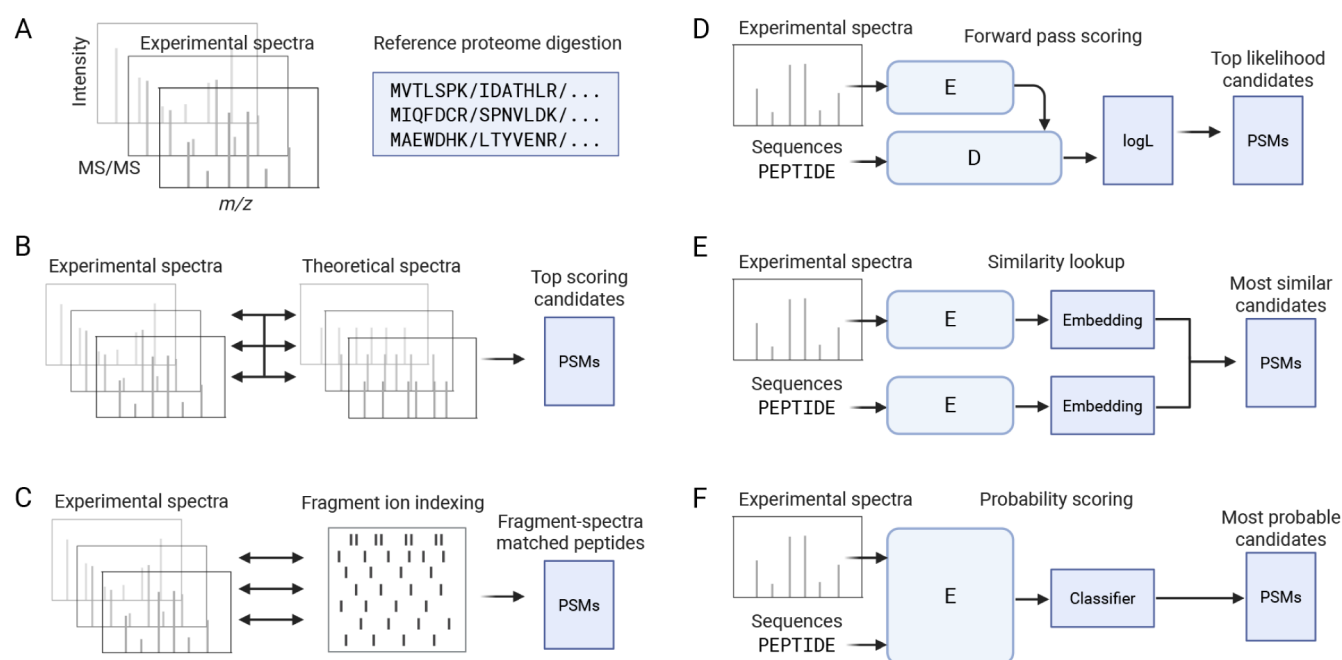


Figure 1. Overview of present and emerging workflows for spectrum identification from tandem mass spectra. **A.** An LC-MS/MS experiment database search starts with experimental tandem mass spectra that will be later searched against an *in silico* digestion of a reference proteome. **B.** In classical database search, experimental spectra are compared to theoretical spectrum generated by calculation of product ions for candidate peptides. **C.** Fragment-ion indexing precomputes and stores all theoretical fragment ions, enabling rapid lookup of candidate peptides. **D.** *De novo* sequencing predicts peptide sequences directly from spectra, independent of the database, but can also be used to score peptides from a theoretical database digest. **E.** encoder, **D:** decoder, logL: log-likelihood. **E.** Biencoder models embed spectra and peptides separately into a shared vector space, where approximate nearest-neighbor (ANN) search retrieves the most similar candidates. **F.** Cross-encoder models jointly process spectra and peptide sequences within a neural network to produce a matching score.

properties. Specifically, we analyze six paradigms: classical database search, fragment-ion indexing, cross-encoder scoring, classical search with neural rescoring, biencoder retrieval with approximate nearest neighbors, and *de novo* sequencing followed by encoder–decoder or cross-encoder scoring. Our goal is to provide a unified analytical framework that places classical and emerging spectrum identification strategies into a common computational context. While many of these methods scale linearly in S and/or P , the critical distinction lies in where that dependence appears and how this affects practical feasibility.

By formally juxtaposing these approaches, and by providing a framework that allows others to perform similar comparisons under their own assumptions, we aim to make their structural differences and trade-offs directly comparable. We hope this framework can serve as a guide for selecting and developing scalable strategies in an era of massive data sets, huge databases, and increasingly powerful neural models. We argue that trained neural scoring models may progressively assume a more central role in database search as data scale and model capacity expand, provided that computational and statistical constraints are carefully addressed.

PRESENT AND POTENTIAL METHODS FOR DATABASE SEARCH

The computational task of spectrum identification can be formulated as scoring experimental spectra against a large reference set of peptide sequences.²⁹ Let S denote the number of experimental tandem mass spectra, and P the number of peptides in the search database. For each spectrum, a subset of peptides is selected as candidates and scored based on the match between observed and theoretical fragments.

In classical search engines, candidate selection is guided by precursor mass, such that only a fraction ρP of the database needs to be considered for each spectrum, where $\rho \in (0, 1]$ denotes the proportion of peptides whose theoretical precursor masses fall within the specified precursor tolerance window around the experimental precursor mass, with $\rho \ll 1$ in practice. Other approaches have recently been proposed for database search as well (core methods summarized in Figure 1). Across different strategies, the scaling of computational cost in S and P ultimately determines their feasibility for data sets and databases of increasing sizes.

Here we focus on methods for data-dependent acquisition (DDA), but most of the underlying computational principles would extend naturally to data-independent acquisition (DIA). In DIA, candidate generation is typically performed in a peptide-centric manner with spectral libraries rather than databases, and fragment-ion indexing strategies such as those implemented in PepCentric⁸ play a role analogous to fragment-based search in DDA. Thus, while the technical details differ, the scaling considerations in S and P are closely related.

Classical Database Search

The classical approach has been the standard in proteomics since the 1990s, implemented in search engines such as SEQUEST, Mascot, MS-GF+, X!Tandem, and Andromeda.^{30–34} The core algorithm performs an *in silico* digest of a protein database, computes theoretical fragment ions, and scores each candidate against experimental spectra. By restricting candidates to those within the precursor mass window, typically only a fraction ρ of peptides are compared to each spectrum. Therefore, the computational complexity is

$$O(S \times \rho P)$$

This approach has been extensively optimized through indexing, vectorized scoring, and precursor-based filtering. It remains robust and effective, but runtime grows linearly with S and with P , and effectively quadratically when both increase together. Moreover, the

effective database size P itself grows combinatorially with the number of variable modifications considered, further amplifying the scaling problem.

Typical modern experiments contain S in the range of 10^4 – 10^8 spectra, while protein databases yield P from 10^5 peptides for a single organism to 10^8 – 10^9 when including multiple organisms, proteoforms, or variable modifications. The fraction $\rho \in (0, 1]$ represents peptides whose precursor masses fall within the tolerance window of a given spectrum and is usually very small, on the order of 10^{-3} – 10^{-5} for typical mass tolerances (10–20 ppm).

Fragment-Ion Indexing

To reduce runtime dependence on database size, fragment-ion indexing precomputes all theoretical fragments from the peptide database and stores them in an inverted index. At query time, each observed spectrum is decomposed into fragment ions, which are looked up in the index to rapidly identify candidate peptides. This strategy, popularized by modern engines such as MSFragger,^{35,36} and also implemented in Sage, Comet and Tide,^{37–39} replaces repeated on-the-fly digestion with direct index lookups. The overall computational complexity can be expressed as

$$O(P + S)$$

where the first term corresponds to index construction, scaling with the number of peptides P and the constant cost of fragment ions per peptide F , and the second term reflects query-time search over S spectra, with the constant cost of index lookups L and number of fragment comparisons per spectrum M . Importantly, runtime is independent of P at query time, though memory requirements are high since all fragment ions (including decoys) must be stored. This approach has proven effective in practice, enabling large-scale searches where classical engines would otherwise become prohibitively slow.

De Novo Sequencing (Encoder–decoder)

De novo peptide sequencing predicts peptide sequences directly from spectra, bypassing the need for a reference database. Recent encoder–decoder transformer models, including Casanovo, InstaNovo, ContraNovo and π -PrimeNovo have greatly improved *de novo* sequencing accuracy,^{20,25–27} making it competitive with database search in some contexts.⁴⁰ The computational complexity scales linearly with the number of spectra,

$$O(S)$$

with a relatively large constant factor given by the per-spectrum neural decoding cost, C_{dn} , dependent on model architecture. In autoregressive decoding models, this per-spectrum inference cost scales linearly with peptide length, as amino acid tokens are generated sequentially. In contrast, nonautoregressive or parallel decoding strategies can predict all positions simultaneously, removing this sequential dependency and reducing sensitivity to peptide length. While this distinction affects practical runtime, it does not alter the overall linear scaling in the number of spectra S considered in our asymptotic framework. This decoding formulation avoids dependence on database size P , which makes it attractive when P is very large or poorly defined, such as in metaproteomics or proteogenomics.

In principle, a *de novo* model can also be applied to score complete peptide sequences obtained from a theoretical digestion of a reference database, extending its role beyond sequence generation to candidate evaluation. However, current *de novo* approaches are much slower than classical database search, often require higher spectral quality with rich fragment ion coverage, and remain less accurate overall. Moreover, the predicted sequences do not necessarily correspond to peptides present in a given database P , which can complicate downstream biological interpretation. *De novo* predictions may therefore be used in combination with rescoring or database matching to improve robustness and calibration.

If we use a *de novo* sequencing model to directly score peptides from a database rather than decode the sequence from scratch, precursor-based filtering can be applied to the candidate search space.

In this setting, only ρP candidate peptides fall within the mass tolerance window for each spectrum, reducing the effective cost to

$$O(S \times \rho P)$$

where the constant factor C_{score} reflects the forward-pass cost of evaluating a candidate peptide with the model rather than performing full sequence decoding. In practice, this scoring pass is substantially cheaper than full autoregressive decoding ($C_{\text{score}} \ll C_{\text{dn}}$). For example, under the hardware assumptions described in the "Comparative Scaling Analysis" section, full *de novo* decoding typically requires $C_{\text{dn}} \approx 2$ – 5×10^{-4} s per spectrum on a modern GPU (corresponding to $\sim 2,000$ – $5,000$ decoded spectra/s), whereas a spectrum–peptide scoring pass using a shared encoder or cross-attention head may require only $C_{\text{score}} \approx 1$ – 5×10^{-6} s per pair. As a result, evaluating ρP candidates via scoring is often substantially cheaper than performing full decoding. In practice, $\rho P \cdot C_{\text{score}} < C_{\text{dn}}$, making this formulation considerably more tractable than full *de novo* decoding while still utilizing the same underlying model architecture for scoring.

Cross-Encoder Joint Scoring

A cross-encoder directly scores spectrum–peptide pairs by jointly encoding both inputs within a shared neural architecture that allows direct interaction between spectrum and peptide representations.⁴¹ In practice, this is most commonly implemented using transformer-based models with cross-attention mechanisms, but the paradigm itself is not restricted to transformers. In such models, the spectrum representation and peptide tokens are concatenated and processed together, allowing the network to learn fine-grained interactions between fragment ions and peptide residues. This formulation is closely related to retrieval models in natural language processing, and differs from *de novo* encoder–decoder models in that it evaluates a fixed candidate peptide sequence rather than generating tokens autoregressively.

For database search, candidate peptides are first restricted by precursor mass tolerance, yielding ρP candidates per spectrum. The cross-encoder then computes a match score for each candidate, leading to a computational complexity of

$$O(S \times \rho P)$$

While this has the same asymptotic form as classical search, the constant factor is substantially larger: each candidate requires a full forward pass through a transformer model, rather than a lightweight dot product. As a result, exhaustive cross-encoder search remains prohibitively expensive for large P or S . Nevertheless, cross-encoders are attractive due to their accuracy,^{41,42} and they are natural candidates for rescoring frameworks where the candidate set is reduced by faster methods. Training can be performed on positive peptide–spectrum matches and decoy negatives, using cross-entropy or margin ranking losses, for example relative to Xcorr scores.

Classical Search with Neural Rescoring

A more tractable way to utilize powerful neural models is to retain the speed of classical candidate generation while applying rescoring to a limited set of top matches. In established proteomics terminology, rescoring typically refers to statistical discrimination frameworks applied to already generated PSMs in order to improve separation between target and decoy distributions at a given FDR threshold. In contrast, in the present framework we use "rescoring" to denote the application of a learned spectrum–peptide scoring model to rescore the best peptide candidates for a spectrum, with the potential to refine or alter the peptide assignment itself. In this formulation, rescoring is a strategy to restrict expensive inference to a narrowed search space. Search engines first filter candidates by precursor mass, yielding ρP peptides per spectrum, and then compute a fast score such as Xcorr. The best p candidates are retained, of which the top $K \leq p$ may be reevaluated by a more powerful model, such as a cross-encoder or encoder–decoder. The computational cost is

$$O(S \times (\rho P + K))$$

Table 1. Estimated Computational Characteristics of Six Spectrum Identification Strategies at a Fixed Use Case ($S = 10^6$ Spectra, $P = 10^8$ Peptides, $\rho = 10^{-4}$)^a

Method	Computational complexity	Build time (approx.)	Wall time (approx.)	GPU usage	Memory (approx.)
Classical database search	$O(S \times \rho P)$	—	~4–6 h	None/optional (scoring acceleration)	10–20 GB (database + buffers)
Fragment-ion indexing (CPU)	$O(P + S)$	~2–4 h	~1–2 h	CPU-based (multithreaded)	100–300 GB (fragment index)
Classical + neural rescoring	$O(S \times (\rho P + K))$	—	~6–8 h total (+1–2 h rescoring)	1 GPU (rescoring)	~20 GB (database + model)
<i>De novo</i> + rescoring	$O(S \times (1 + K))$	—	~10–15 h (inference bound)	1–2 GPUs (decoding + rescoring)	16–24 GB (model + tensors)
Cross-encoder search	$O(S \times \rho P)$	—	~2–12 days	GPU required (per-pair evaluation)	<20 GB (model weights)
Biencoder + ANN retrieval	$O(S \times (\log P + K))$	~4–6 h (embedding + ANN index)	~2–3 h (search)	1 GPU (embedding + retrieval)	40–80 GB (embedding index, 128–256D)

^aWall times and memory footprints are approximate and based on modeled assumptions for a dual-socket CPU server (32–64 Cores) and an NVIDIA A100 GPU (80 GB VRAM). Fragment-ion indexing assumes ~24 fragment ions per peptide and 5×10^6 fragment postings/S; bi-encoder indexing assumes 128–256d embeddings at 10^4 peptides/S with ~20% ANN overhead. Cross-encoder and encoder–decoder inference are modeled at 10^4 and 2×10^3 forward passes/S, respectively.

In this scheme, only the top-ranked candidates from the initial scoring stage are passed through the neural model for evaluation, ensuring that the expensive inference is applied sparingly. The value of K thus directly balances accuracy against runtime: increasing K may improve sensitivity but increases computational load, whereas smaller K keeps runtime low but risks missing true positives. Because the expensive model is confined to a small candidate set, this strategy combines efficient candidate reduction with accuracy gains. It extends a long tradition of rescoring in proteomics, from semisupervised learning to modern neural classifiers,^{4,5,15} and is currently among the most practical ways to integrate machine learning into database search.

De Novo Sequencing with Rescoring

De novo predictions can also be combined with database information through rescoring strategies. In this setup, an encoder–decoder model first proposes one or several peptide sequences directly from each spectrum. These predictions are then aligned to the database by identifying peptides within precursor mass tolerance that are most similar to the *de novo* output. Similarity can be defined in multiple ways, including direct sequence edit distance, overlap of theoretical versus observed fragment ions, similarity of physicochemical properties, or proximity in a latent embedding space. A correlation score or a secondary model, such as a cross-encoder, can then be applied to rescore this reduced candidate set. The resulting complexity is

$$O(S \times (1 + K))$$

where the first term corresponds to one decoding pass per spectrum (a constant per-spectrum inference cost C_{dn}), and K is the number of candidates rescored. As with classical rescoring, the choice of K governs the trade-off between runtime and sensitivity. Encoder–decoder models may be autoregressive or nonautoregressive, with the latter offering potential gains in inference speed and cost. This hybrid formulation exploits the ability of *de novo* models to generate plausible sequences independently of the database, while still constraining results to peptides present in P .

Biencoder Retrieval with Approximate Nearest Neighbors

Biencoders embed spectra and peptides into a shared vector space,^{23,43} enabling efficient retrieval of candidate peptides through approximate nearest-neighbor (ANN) search.^{44,45} ANN methods are designed to rapidly find vectors in a large database that are closest in distance (e.g., cosine similarity or Euclidean distance) to a query vector, without exhaustively comparing against all entries. They rely on specialized data structures such as hierarchical graphs, trees, or quantization schemes that allow sublinear retrieval in database size. Unlike cross-encoders, which jointly process both inputs, biencoders compute embeddings independently and score them by similarity in the embedding space. This means that peptide embeddings can be precomputed, stored in an index, and then queried efficiently by

comparing spectrum embeddings only at search time. Conceptually, this is similar to fragment-ion indexing in that a precomputed database representation is used to avoid repeated on-the-fly scoring, but here the representation is neural embeddings rather than fragment lists. As such, the computational complexity is

$$O(S \times (\log P + K))$$

where $\log P$ reflects the cost of ANN retrieval and K the number of top candidates retained for further scoring. The main expense lies in building and storing the peptide embedding index, which must fit in memory and support fast retrieval. This approach offers an attractive avenue for scaling database search with neural models, particularly as large embedding models and high-performance ANN algorithms reach maturity.

COMPARATIVE SCALING ANALYSIS

To enable a concrete comparison of the computational trade-offs discussed above, we estimated practical wall-time and memory requirements for each method under representative modern hardware conditions. Our goal was not to benchmark specific software implementations, but to approximate relative scaling behavior using realistic compute assumptions typical for contemporary research environments.

We assume a modern compute environment representative of current proteomics workflows. For CPU-based methods, we model a multicore server processor with vectorized scoring kernels sustaining $\sim 2 \times 10^6$ spectrum–peptide comparisons per second. GPU acceleration is modeled on a datacenter-class device (e.g., NVIDIA A100) sustaining $\sim 2 \times 10^7$ dot-product evaluations per second. Cross-encoder inference is assumed at $\sim 10^4$ spectrum–peptide pairs per second, while encoder–decoder *de novo* decoding achieves $\sim 2 \times 10^3$ spectra per second. ANN retrieval is modeled as logarithmic in P , with ~ 0.2 ms per query and an additional cost for reranking. Index build times and memory requirements are estimated from published benchmarks and representative implementations.

To facilitate further and customizable exploration, the accompanying code and visualization scripts are available in an open repository (https://github.com/instadeepai/database_search_scaling), enabling readers to reproduce our calculations, alter all relevant parameters, and examine scaling behavior under their own assumptions regarding database size, number of spectra, hardware capabilities, or model inference speeds. The repository will be maintained and extended to

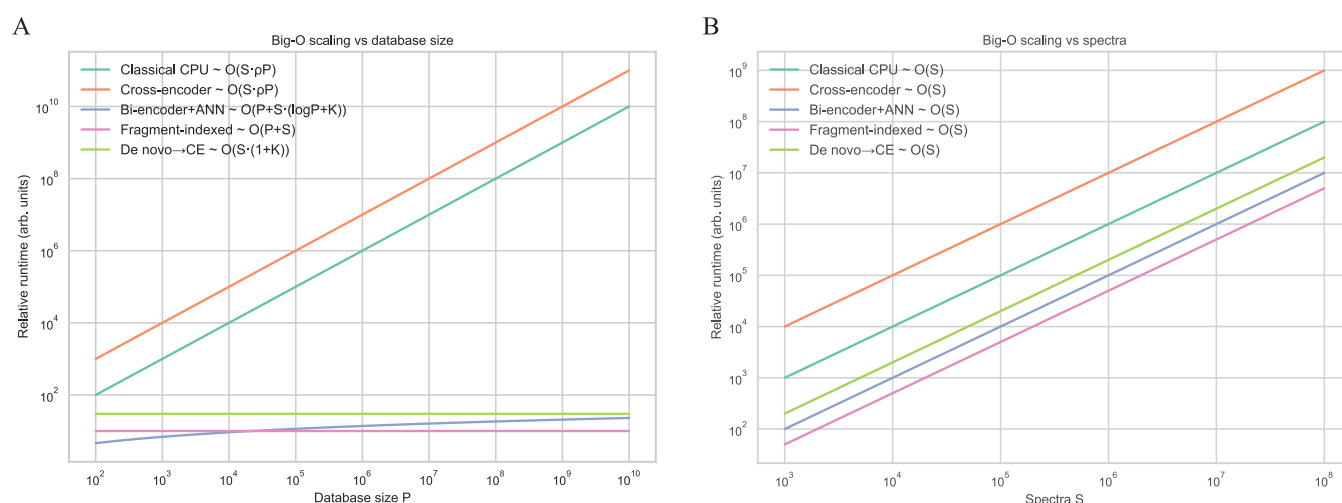


Figure 2. A. Asymptotic scaling of runtime with database size P for fixed spectra $S = 10^6$. Classical search and cross-encoder scoring scale linearly with P , while fragment-ion indexing and *de novo* with rescoring are independent of P . Biencoder retrieval grows only logarithmically with P , offering the most favorable scaling at large database sizes. B. Asymptotic scaling of runtime with the number of spectra S for fixed database size $P = 4 \times 10^8$. All methods scale linearly with S , but differ substantially in constant factors. Cross-encoder scoring is orders of magnitude slower, while fragment-ion indexing and biencoder retrieval show reduced overhead.

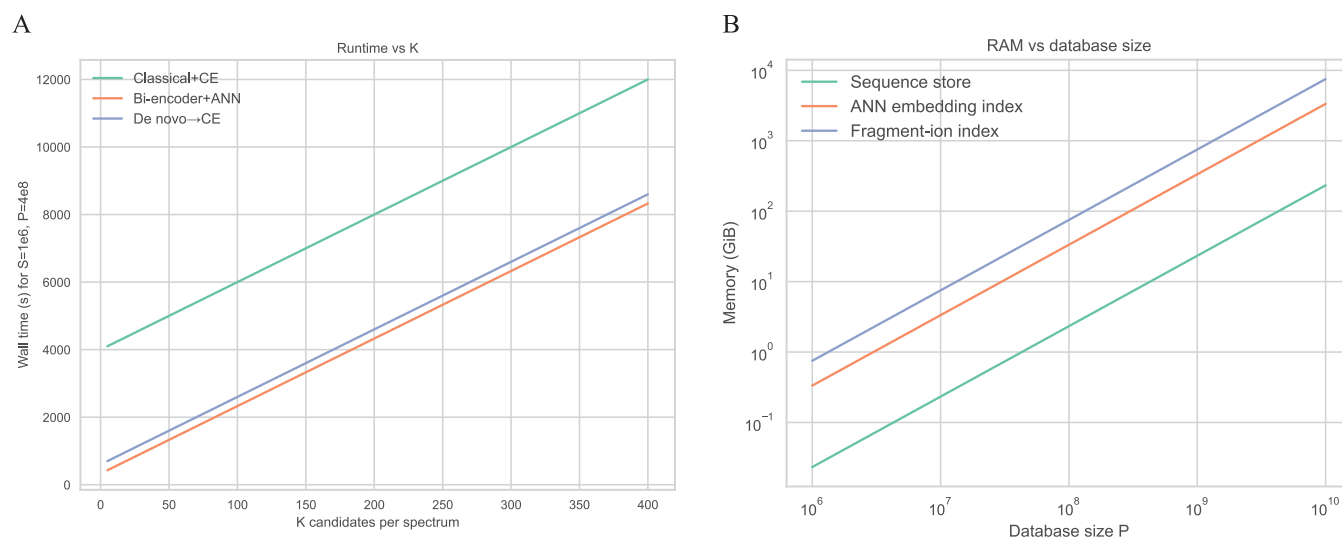


Figure 3. A. Runtime as a function of the number of candidates K rescored per spectrum, for $S = 10^6$ and $P = 4 \times 10^8$. Classical search with cross-encoder rescoring, biencoder retrieval, and *de novo* with rescoring all scale linearly with K , but differ in baseline cost. Efficient candidate reduction is critical for practical rescoring. B. Estimated memory requirements as a function of database size P . Classical search requires only the sequence database, whereas fragment-ion indexing and ANN-based biencoder retrieval impose substantial memory demands. Fragment-ion indices grow fastest due to storage of all theoretical fragments including decoys.

include emerging architectures and updated hardware models as they become available.

Asymptotic Scaling in S and P

These six approaches differ fundamentally in how runtime scales with the number of spectra (S) and peptides in the database (P) (Table 1). Classical search and cross-encoder scoring both scale as $O(S \times pP)$, since candidates are restricted by precursor mass tolerance, but the constant factors differ dramatically. In fragment ion indexing search, the dominant operation is vectorized fragment matching, which is computationally inexpensive. In contrast, cross-encoders must perform a forward pass of a large neural network for each spectrum–peptide pair, making them several orders of magnitude slower despite the same asymptotic complexity. Fragment-ion indexing and *de novo* peptide prediction with rescoring remove the

explicit dependence on P , while biencoder retrieval scales only as $O(S \log P)$, offering the most favorable behavior at very large databases (Figure 2A). Classical search remains efficient for small to moderate P , but becomes prohibitive as databases approach 10^8 peptides due to its linear scaling in both S and P . Fragment-ion indexing avoids P -dependence at query time by precomputing all theoretical fragments, but requires substantial preprocessing and large memory to store the index. Biencoder retrieval instead precomputes peptide embeddings, allowing sublinear candidate lookup through approximate nearest-neighbor (ANN) search, and thus becomes advantageous at large P . Hybrid *de novo* strategies depend only on S , since predictions are independent of database size, while pure cross-encoder search (or encoder–decoder models applied ex-

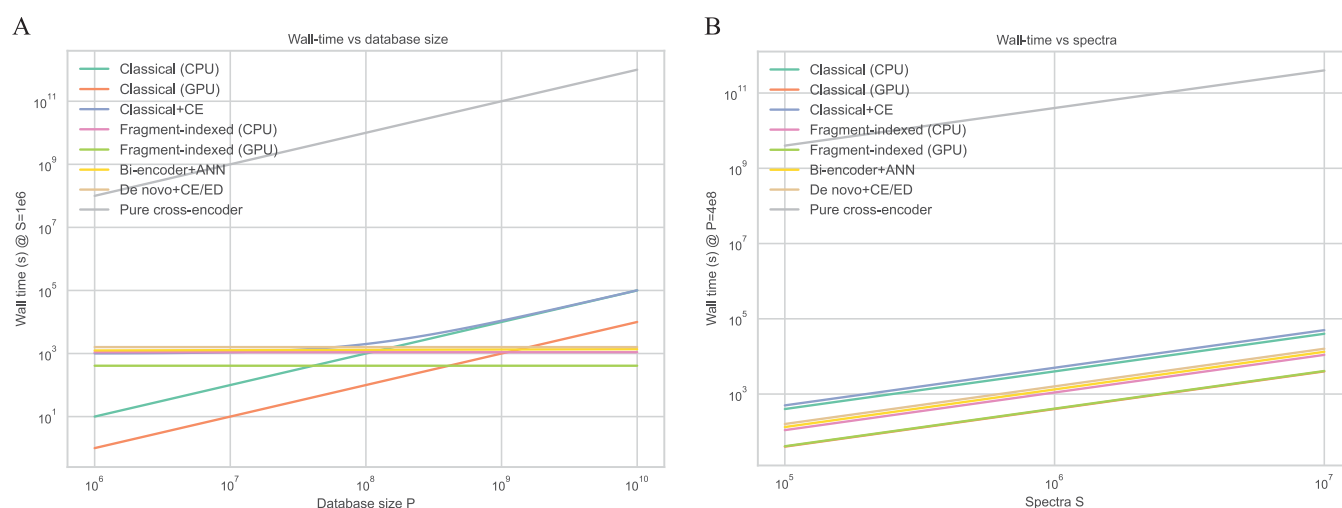


Figure 4. A. Estimated wall time versus database size P at fixed $S = 10^6$ spectra. Classical search is fast at moderate P but grows linearly, while fragment-ion indexing and biencoder retrieval remain nearly flat. *De novo* with rescoring is independent of P , and cross-encoder scoring becomes intractable at large P . B. Estimated wall time versus number of spectra S at fixed $P = 4 \times 10^8$. All methods scale linearly with S , but cross-encoder scoring is prohibitive even at moderate data set sizes. Fragment-ion indexing and biencoder retrieval maintain practical runtimes, while *de novo* with rescoring scales favorably but has higher per-spectrum cost.

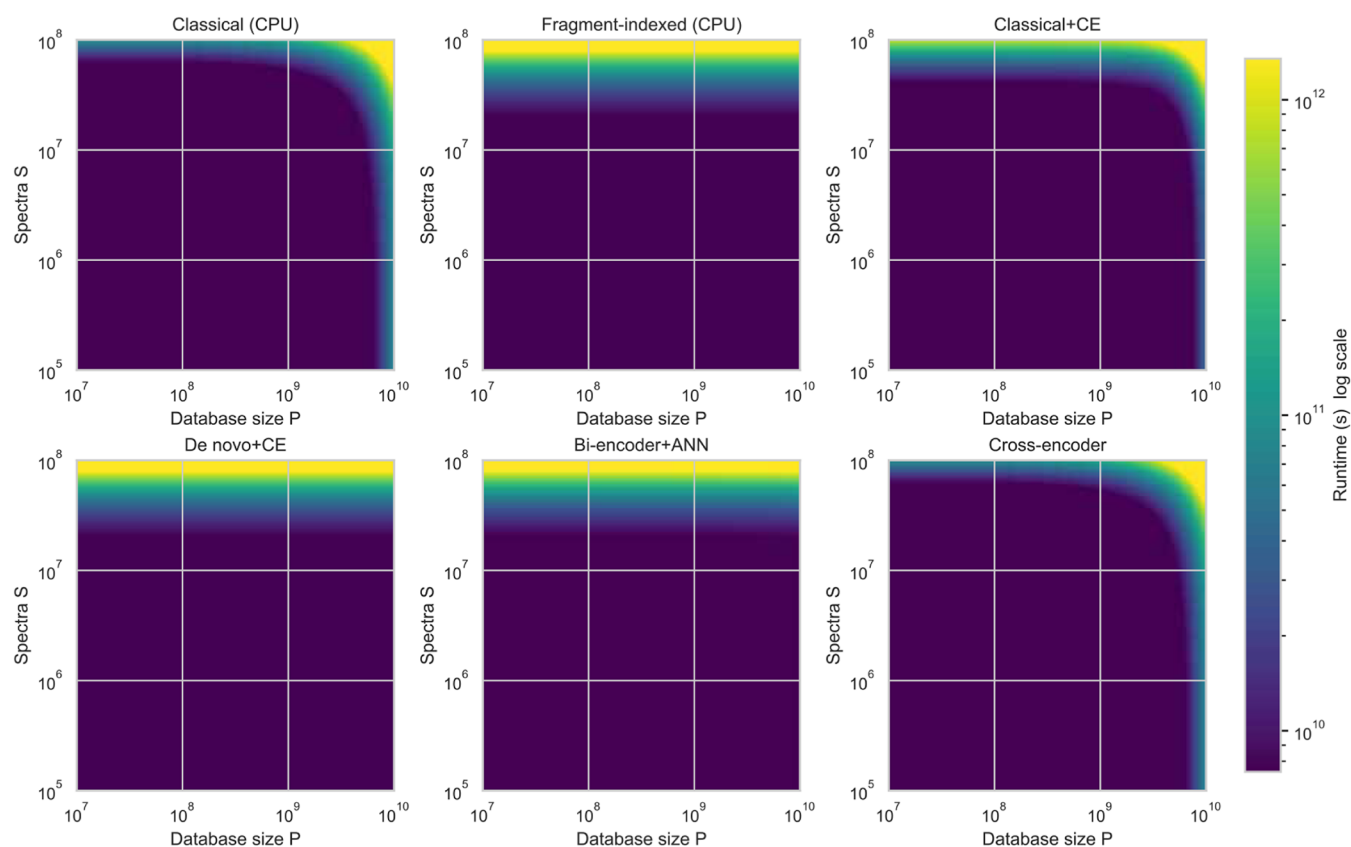


Figure 5. Heatmaps of runtime scaling as a function of both S and P across six approaches. Classical search becomes prohibitive at very large P , whereas fragment-ion indexing and biencoder retrieval remain tractable. *De novo* with rescoring depends only on S , while pure cross-encoder scoring is infeasible at scale. Color scale indicates $\log_{10}$ runtime (arbitrary units).

haustively) is infeasible at scale due to the cost of running the network over the full search space.

All approaches scale linearly with S , but their constant factors differ substantially (Figure 2B). Cross-encoder scoring is orders of magnitude slower because every candidate requires a full forward pass of a large model, whereas fragment-ion indexing and biencoder retrieval amortize computation by

reusing precomputed representations and efficient lookups. Hybrid *de novo* with rescoring has a higher per-spectrum cost than classical search, because generating and evaluating predictions involves network inference, but this overhead is manageable when combined with rescoring of a limited candidate set. Pure *de novo* sequencing is not considered further here, as its much larger inference overhead makes it

impractical compared to hybrid strategies where predictions are constrained and rescored against the database.

Memory and Rescoring Overhead

The cost of rescoring grows linearly with the number of candidates K per spectrum. Classical search with cross-encoder rescoring, biencoder retrieval, and *de novo* with rescoring all scale as $O(S \times K)$, but differ in baseline cost. Efficient candidate filtering is therefore essential to keep K modest, especially in neural rescoring contexts (Figure 3A).

When comparing memory requirements, classical search is memory-light. It requires only the peptide sequence database (~ 2.5 GiB for 10^8 peptides), whereas ANN indices for biencoder retrieval require ~ 30 – 50 GiB at $P = 10^8$, depending on embedding dimension and index overhead (Figure 3B). Fragment-ion indexing grows fastest, exceeding 100 GiB at $P = 10^8$ when decoys are included. This makes memory the key constraint for fragment-based approaches, although the index can be sharded. The build time for fragment ion indexing is faster, however it requires large memory consumption. In contrast, biencoder embeddings have a slower build time but moderate memory consumption.

Estimated Wall Times and Practical Limits

At $S = 10^6$ spectra, classical search remains fast up to $P \approx 10^7$, but beyond this point fragment-ion indexing and biencoder retrieval become superior (Figure 4A). The reason is that classical search scales linearly in both S and P , so runtime grows quickly as databases expand. Classical search begins to lag at $P \gtrsim 10^8$ because it must score an ever-growing number of candidates per spectrum. *De novo* with rescoring is independent of P , since candidates are generated directly from spectra, but has higher absolute cost per spectrum due to the inference overhead of sequence prediction as discussed above. Pure cross-encoder search is infeasible: at $S = 10^6$ and $P = 10^8$, it would require 10^{14} evaluations, corresponding to nearly $\sim 1,000$ years of wall time even at millisecond inference rates.

For very large data sets, all methods grow linearly with S , but their practical limits differ (Figure 4B). Fragment-ion indexing and biencoder retrieval remain tractable at $P = 4 \times 10^8$, since both avoid explicit dependence on P at query time. *De novo* with rescoring also scales favorably, but its throughput is ultimately bounded by the decoding cost of generating candidate sequences. Cross-encoder scoring remains impractical in exhaustive search, yet is highly effective in rescoring contexts, where only K top candidates are evaluated and the high model cost is applied sparingly.

OUTLOOK AND PERSPECTIVES

Neural network models offer clear opportunities to improve peptide detection, but their computational cost remains a barrier for routine use at scale. Our analysis highlights where each approach is tractable and where bottlenecks emerge, with several directions already showing promise in scaling and implementation (Figure 5).

At first glance, hybrid pipelines that retain the efficiency of classical candidate generation while introducing neural rescoring appear immediately practical. Such strategies can exploit cross-encoder or encoder–decoder models for improved peptide scoring calibration and potentially rescoring or PTM localization without prohibitive runtime.

In addition, biencoder embeddings with approximate nearest-neighbor retrieval represent a scalable path for large

databases. Techniques such as quantization, hierarchical indices, distillation, and GPU-native ANN libraries may further reduce memory and inference cost, making this approach viable even at proteogenomic scale.

Importantly, future hardware and algorithm developments are likely to shift practical boundaries.⁴⁶ Multi-GPU inference, tensor processing units, and mixed-precision models can reduce per-spectrum inference time by orders of magnitude,^{47,48} enabling neural methods that are currently out of reach.

As with all learned models, careful evaluation of generalization, calibration, and overfitting is essential. These considerations are not unique to neural architectures, but proper validation and statistical control remain critical as model capacity increases.

Finally, benchmarking and open-source implementations will be critical. Systematic evaluation on realistic large-scale data sets is needed to quantify accuracy gains relative to cost, and to guide the community toward balanced trade-offs. In this way, peptide database search can follow the trajectory of other AI domains, where hybrid retrieval and neural scoring are already standard practice.

Overall, it is conceivable that in the near future classical search engines and neural embedding models are not competing paradigms but complementary building blocks. The challenge is to design workflows that exploit their respective strengths to deliver both scalability and accuracy in the era of massive data sets. The central question is therefore not whether machine learning can be incorporated into database search, as such techniques are already established in the field, but whether learned scoring models may progressively replace or augment classical similarity functions at the peptide–spectrum scoring level.

AUTHOR INFORMATION

Corresponding Author

Konstantinos Kalogeropoulos – Department of Biotechnology and Biomedicine, Technical University of Denmark, Kongens Lyngby 2800, Denmark; Center for Translational Protein Design, Technical University of Denmark, Kongens Lyngby 2800, Denmark; Department of Bionanoscience, Delft University of Technology, Delft 2629 HZ, Netherlands; Kavli Institute of Nanoscience, Delft 2629 HZ, Netherlands; orcid.org/0000-0003-3907-9281; Email: konka@dtu.dk

Authors

Jeroen Van Goey – InstaDeep Ltd, London W2 1AY, UK

Timothy P. Jenkins – Department of Biotechnology and Biomedicine, Technical University of Denmark, Kongens Lyngby 2800, Denmark; Center for Translational Protein Design, Technical University of Denmark, Kongens Lyngby 2800, Denmark; orcid.org/0000-0003-2979-5663

Kevin Michael Eloff – InstaDeep Ltd, London W2 1AY, UK

Complete contact information is available at:

<https://pubs.acs.org/10.1021/acs.jproteome.5c01153>

Author Contributions

K.K. conceived and wrote the original draft of the manuscript. K.K. and J.V.G. conducted the scaling analysis. K.M.E. and T.P.J. provided feedback and edited the manuscript. All authors reviewed the manuscript and approved its final version.

Notes

Declaration of generative AI and AI-assisted technologies in the writing process: During the preparation of this work, the authors used GPT-5.1 in order to improve language and readability. After using this tool, the authors reviewed and edited the content as needed and take full responsibility for the content of the manuscript.

The authors declare the following competing financial interest(s): K.E and J.V.G. are employees of InstaDeep, 5 Merchant Square, London, UK. The other authors declare no competing interests.

ACKNOWLEDGMENTS

K.K. is supported by a Novo Nordisk Foundation Young Investigator Award (grant no. NNF16OC0020670) and a postdoctoral fellowship grant from the Independent Research Fund Denmark (grant no. 4257-00010B). We thank the entire InstaNovo team for their valuable input and feedback during this study. Figure ¹ was made with <https://Biorender.com>.

REFERENCES

- Eng, J. K.; Searle, B. C.; Clauser, K. R.; Tabb, D. L. A Face in the Crowd: Recognizing Peptides Through Database Search. *Mol. Cell. Proteomics* **2011**, *10* (11), R111.009522.
- Verheggen, K.; Ræder, H.; Berven, F. S.; Martens, L.; Barsnes, H.; Vaudel, M. Anatomy and evolution of database search engines—a central component of mass spectrometry based proteomic workflows. *Mass Spectrom. Rev.* **2020**, *39*, 292–306.
- Chick, J. M.; Kolippakkam, D.; Nusinow, D. P.; Zhai, B.; Rad, R.; Huttlin, E. L.; Gygi, S. P. A mass-tolerant database search identifies a large proportion of unassigned spectra in shotgun proteomics as modified peptides. *Nat. Biotechnol.* **2015**, *33*, 743–749.
- Käll, L.; Canterbury, J. D.; Weston, J.; Noble, W. S.; MacCoss, M. J. Semi-supervised learning for peptide identification from shotgun proteomics datasets. *Nat. Methods* **2007**, *4*, 923–925.
- Ma, K.; Vitek, O.; Nesvizhskii, A. I. A statistical model-building perspective to identification of MS/MS spectra with PeptideProphet. *BMC Bioinf.* **2012**, *13*, S1.
- Yu, F.; Teo, G. C.; Kong, A. T.; Haynes, S. E.; Avtonomov, D. M.; Geisler, D. J.; Nesvizhskii, A. I. Identification of modified peptides using localization-aware open search. *Nat. Commun.* **2020**, *11*, 4065.
- Nesvizhskii, A. I. Proteogenomics: concepts, applications and computational strategies. *Nat. Methods* **2014**, *11*, 1114–1125.
- Yu, F.; Kong, A. T.; Hsiao, Y.; Nesvizhskii, A. I. PepCentric Enables Fast Repository-Scale Proteogenomics Searches. *bioRxiv* **2025**.
- Xian, F.; Brenek, M.; Krisp, C.; Urbauer, E.; Ravi Kumar, R. K.; Aguan, D.; Srikumar, T.; Liu, Q.; Barry, A. M.; Ma, B.; et al. Ultra-sensitive metaproteomics redefines the dark metaproteome, uncovering host-microbiome interactions and drug targets in intestinal diseases. *Nat. Commun.* **2025**, *16*, 6644.
- Meyer, J. G. Deep learning neural network tools for proteomics. *Cell Rep. Methods* **2021**, *1*, 100003.
- Ma, C.; Ren, Y.; Yang, J.; Ren, Z.; Yang, H.; Liu, S. Improved Peptide Retention Time Prediction in Liquid Chromatography through Deep Learning. *Anal. Chem.* **2018**, *90*, 10881–10888.
- Bouwmeester, R.; Gabriels, R.; Hulstaert, N.; Martens, L.; Degroove, S. DeepLC can predict retention times for peptides that carry as-yet unseen modifications. *Nat. Methods* **2021**, *18*, 1363–1369.
- Gessulat, S.; Schmidt, T.; Zolg, D. P.; Samaras, P.; Schnatbaum, K.; Zerweck, J.; Knaute, T.; Reichenberger, J.; Delanghe, B.; Huhmer, A.; Reimer, U.; Ehrlich, H.-C.; Aiche, S.; Kuster, B.; Wilhelm, M. Prosit: proteome-wide prediction of peptide tandem mass spectra by deep learning. *Nat. Methods* **2019**, *16*, 509–518.
- Yang, K. L.; Yu, F.; Teo, G. C.; Li, K.; Demichev, V.; Ralser, M.; Nesvizhskii, A. I. MSBooster: improving peptide identification rates using deep learning-based features. *Nat. Commun.* **2023**, *14*, 4539.
- Demichev, V.; Messner, C. B.; Vernardis, S. I.; Lilley, K. S.; Ralser, M. DIA-NN: neural networks and interference correction enable deep proteome coverage in high throughput. *Nat. Methods* **2020**, *17*, 41–44.
- Zeng, W.-F.; Zhou, X.-X.; Willems, S.; Ammar, C.; Wahle, M.; Bludau, I.; Voytik, E.; Strauss, M. T.; Mann, M. AlphaPeptDeep: a modular deep learning framework to predict peptide properties for proteomics. *Nat. Commun.* **2022**, *13*, 7238.
- Liu, Z.; Liu, P.; Sun, Y.; Nie, Z.; Zhang, X.; Zhang, Y.; Chen, Y.; Guo, T. DIA-BERT: pre-trained end-to-end transformer models for enhanced DIA proteomics data analysis. *Nat. Commun.* **2025**, *16*, 3530.
- Ananth, V.; Sanders, J.; Yilmaz, M.; Wen, B.; Oh, S.; Noble, W. S. A learned score function improves the power of mass spectrometry database search. *Bioinformatics* **2024**, *40*, i410–i417.
- Frejino, M.; et al. Unifying the analysis of bottom-up proteomics data with CHIMERYS. *Nat. Methods* **2025**, *22*, 1017–1027.
- Jin, Z.; Xu, S.; Zhang, X.; Ling, T.; Dong, N.; Ouyang, W.; Gao, Z.; Chang, C.; Sun, S. ContraNovo: A Contrastive Learning Approach to Enhance De Novo Peptide Sequencing. *arXiv* **2023**.
- Klammer, A. A.; Park, C. Y.; Noble, W. S. Statistical calibration of the SEQUEST XCorr function. *J. Proteome Res.* **2009**, *8*, 2106–2113.
- Nesvizhskii, A. I. A survey of computational methods and error rate estimation procedures for peptide and protein identification in shotgun proteomics. *J. Proteomics* **2010**, *73*, 2092–2123.
- Altenburg, T.; Muth, T.; van Zalm, P.; Steen, H.; Renard, B. Y. Foundation Model Enables Interpretable Open and Error-Tolerant Searching for Mass Spectrometry-Based Proteomics. *bioRxiv* **2025**.
- Xiong, P.; Xu, H.; Zheng, H. Supervised Contrastive Learning Leads to More Reasonable Spectral Embeddings. *Anal. Chem.* **2025**, *97*, 20137–20146.
- Yilmaz, M.; Fondrie, W. E.; Bittremieux, W.; Melendez, C. F.; Nelson, R.; Ananth, V.; Oh, S.; Noble, W. S. Sequence-to-sequence translation from mass spectra to peptides with a transformer model. *Nat. Commun.* **2024**, *15*, 6427.
- Eloff, K.; et al. InstaNovo enables diffusion-powered de novo peptide sequencing in large-scale proteomics experiments. *Nat. Mach. Intell.* **2025**, *7*, 565–579.
- Zhang, X.; Ling, T.; Jin, Z.; Xu, S.; Gao, Z.; Sun, B.; Qiu, Z.; Wei, J.; Dong, N.; Wang, G.; et al. π -PrimeNovo: an accurate and efficient non-autoregressive deep learning model for de novo peptide sequencing. *Nat. Commun.* **2025**, *16* (1), 267.
- Straub, G.; Ananth, V.; Fondrie, W. E.; Hsu, C.; Klaproth-Andrade, D.; Riffle, M.; Sanders, J.; Wen, B.; Xu, L.; Yilmaz, M.; et al. Improvements to Casanovo, a deep learning de novo peptide sequencer. *bioRxiv* **2025**.
- Sadygov, R. G.; Cociorva, D.; Yates, J. R. Large-scale database searching using tandem mass spectra: Looking up the answer in the back of the book. *Nat. Methods* **2004**, *1*, 195–202.
- Eng, J. K.; McCormack, A. L.; Yates, J. R. An approach to correlate tandem mass spectral data of peptides with amino acid sequences in a protein database. *J. Am. Soc. Mass Spectrom.* **1994**, *5*, 976–989.
- Perkins, D. N.; Pappin, D. J. C.; Creasy, D. M.; Cottrell, J. S. Probability-based protein identification by searching sequence databases using mass spectrometry data. *Electrophoresis* **1999**, *20*, 3551–3567.
- Kim, S.; Pevzner, P. A. MS-GF+ makes progress towards a universal database search tool for proteomics. *Nat. Commun.* **2014**, *5*, 5277.
- Craig, R.; Beavis, R. C. TANDEM: matching proteins with tandem mass spectra. *Bioinformatics* **2004**, *20*, 1466–1467.
- Cox, J.; Neuhauser, N.; Michalski, A.; Scheltema, R. A.; Olsen, J. V.; Mann, M. Andromeda: a peptide search engine integrated into the MaxQuant environment. *J. Proteome Res.* **2011**, *10*, 1794–1805.

- (35) Kong, A. T.; Leprevost, F. V.; Avtonomov, D. M.; Mellacheruvu, D.; Nesvizhskii, A. I. MSFragger: ultrafast and comprehensive peptide identification in mass spectrometry-based proteomics. *Nat. Methods* **2017**, *14*, 513–520.
- (36) Yu, F.; Deng, Y.; Nesvizhskii, A. I. MSFragger-DDA+ enhances peptide identification sensitivity with full isolation window search. *Nat. Commun.* **2025**, *16*, 3329.
- (37) Lazear, M. R. Sage: An Open-Source Tool for Fast Proteomics Searching and Quantification at Scale. *J. Proteome Res.* **2023**, *22*, 3652–3659.
- (38) McGann, C. D.; Bergstrom, E. J.; Sharma, V.; Heil, L. R.; Yu, Q.; Eng, J. K.; Schweppe, D. K. Comet Fragment-Ion Indexing for Enhanced Peptide Sequencing. *J. Proteome Res.* **2025**, *24*, 3715–3721.
- (39) Kertesz-Farkas, A.; Acquaye, F. L. N. A.; Ostapenko, V.; Locon, R. H.; Lu, Y.; Grant, C. E.; Noble, W. S. Fast and Memory-Efficient Searching of Large-Scale Mass Spectrometry Data Using Tide. *J. Proteome Res.* **2025**, *24*, 4831–4837.
- (40) Bittremieux, W.; Ananth, V.; Fondrie, W. E.; Melendez, C.; Pominova, M.; Sanders, J.; Wen, B.; Yilmaz, M.; Noble, W. S. Deep Learning Methods for De Novo Peptide Sequencing. *Mass Spectrom. Rev.* **2024**.
- (41) Reimers, N.; Gurevych, I. Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks. *arXiv* **2019**.
- (42) Tamber, M. S.; Kazi, S.; Sourabh, V.; Lin, J. Conventional Contrastive Learning Often Falls Short: Improving Dense Retrieval with Cross-Encoder Listwise Distillation and Synthetic Data. *arXiv* **2025**.
- (43) Devlin, J.; Chang, M.-W.; Lee, K.; Toutanova, K. BERT Pre-training of Deep Bidirectional Transformers for Language Understanding. *arXiv* **2018**.
- (44) Zheng, Y.; Ou, Y.; Lex, A.; Phillips, J. M. Visualization of Big Spatial Data using Coresets for Kernel Density Estimates. *IEEE Trans. Big. Data* **2021**, *7* (3), 524–534.
- (45) Bittremieux, W.; Meysman, P.; Noble, W. S.; Laukens, K. Fast open modification spectral library searching through approximate nearest neighbor indexing. *J. Proteome Res.* **2018**, *17*, 3463–3474.
- (46) Kumar, S.; Zambreno, J.; Khokhar, A.; Akram, S.; Saeed, F. FiCOPS: Hardware/Software Co-Design of FPGA Computational Framework for Mass Spectrometry-Based Peptide Database Search. *bioRxiv* **2026**.
- (47) Onoprishvili, T.; Yuan, J.-H.; Petrov, K.; Ingalalli, V.; Khederlarian, L.; Leuchtenmuller, N.; Chandra, S.; Duarte, A.; Bender, A.; Gloaguen, Y. SimMS: a GPUaccelerated cosine similarity implementation for tandem mass spectrometry. *Bioinformatics* **2025**, *41*, btaf081.
- (48) Nayan, M. M. R.; Li, Z.; Ponzina, F.; Pinge, S.; Rosing, T.; Naeemi, A. J. HERP: Hardware for Energy Efficient and Realtime DB Search and Cluster Expansion in Proteomics. *arXiv* **2025**.